LokRozgar AI

Technical System Documentation

Multilingual, voice-enabled employment platform connecting blue-collar workers with local employers.

PROJECT	LokRozgar AI
DOCUMENT	System Documentation
TECHNICAL SCOPE	Frontend · Backend · AI Service · Database
VERSION	1.0.0
DATE	October 2026
AUTHORS	Swarup Pal, Nilam (Backend Lead)
CLASSIFICATION	Technical System Documentation — Implementation Reference

This document is generated from the project's Markdown source of truth. Its technical content is reproduced without modification.

Contents

1. Abstract 4

2. Project Overview 5
- 2.1 Problem Statement 5
- 2.2 Proposed Solution 5
- 2.3 Key Contributions 5

3. System Architecture 6
- 3.1 High-Level Architecture 6
- 3.2 Service Boundaries & Ownership 6
- 3.3 Communication Protocol 7
- 3.4 Deployment Topology 7

4. Technology Stack 9
- 4.1 Frontend 9
- 4.2 Backend (Node/Express) 9
- 4.3 AI Service (Python/FastAPI) 10
- 4.4 Database 10
- 4.5 External Services 10

5. Database Design 11
- 5.1 Entity-Relationship Model 11
- 5.2 Schema Definitions 11
- 5.3 Indexing Strategy 15
- 5.4 Multilingual Data Architecture 15

6. Authentication & Authorization 17
- 6.1 Authentication Flow 17
- 6.2 Token Management 17
- 6.3 Role-Based Access Control 17
- 6.4 Rate Limiting Strategy 18

7. AI & Machine Learning Pipeline 19
- 7.1 Voice Processing Pipeline 19
- 7.2 Profile Extraction Engine 19
- 7.3 Number Grounding Guard 20
- 7.4 Skill Resolution Pipeline 21
- 7.5 Translation Service 21

8. Matching & Ranking System 22
- 8.1 Architecture Overview 22
- 8.2 Six-Factor Matching Algorithm 22
- 8.3 Ranking Engine 24
- 8.4 Recommendation Pipeline 24
- 8.5 Explanation Generator 25

9. Application & Hiring Flow 26
- 9.1 Application Lifecycle 26
- 9.2 Offer Management 26

9.3 Capacity Control & Atomic Hiring 27
9.4 Shift Management 27

10. Job Posting & Management 29
10.1 Job Creation with Auto-Translation 29
10.2 Job Lifecycle State Machine 29
10.3 Geospatial Querying 29

11. Communication System 31
11.1 Real-Time Chat (Socket.io) 31
11.2 Notification System 31

12. Frontend Architecture 32
12.1 Component Hierarchy 32
12.2 Routing & Guards 32
12.3 State Management 33
12.4 Internationalization (i18n) 33
12.5 Voice Profile UI Workflow 34

13. API Reference 35
13.1 Backend REST API 35
13.2 AI Service API 37

14. Payment & Dispute Resolution 38
Payment Flow 38
Dispute Resolution 38

15. Security Architecture 39

16. Testing Strategy 40

17. Performance Considerations 41

18. Future Roadmap 42

19. Glossary 43

Version: 1.0.0
Date: October 2026
Authors: Swarup Pal, Nilam (Backend Lead)
Classification: Technical System Documentation — Implementation Reference

1. Abstract

LokRozgar AI is a multilingual employment platform purpose-built for blue-collar workers and local employers in India's semi-urban and rural labour markets. The platform addresses a fundamental accessibility barrier: the majority of its target demographic speaks Marathi, Hindi, or a mix of both, and may have limited text-literacy. LokRozgar overcomes this through a **voice-first profile creation system** powered by on-device AI inference (MLX Whisper for speech-to-text and Qwen for structured field extraction), **automatic trilingual translation** (via Sarvam AI), and a **six-factor weighted matching algorithm** that considers skill overlap, geospatial proximity, worker availability, pay compatibility, historical ratings, and profile-to-job relevance.

The system is composed of three independently deployable services: a React/Vite frontend, a Node.js/Express backend (the sole owner of MongoDB and all business logic), and a Python FastAPI AI service (responsible for voice processing, translation, matching, ranking, and recommendation). The frontend never communicates with the AI service directly — all AI operations are orchestrated by the backend.

This document provides a complete technical description of the implemented system as of October 2026, including database schemas, algorithms, data flows, API contracts, security mechanisms, and deployment considerations.

2. Project Overview

2.1 Problem Statement

India's informal labour sector employs over 400 million workers, yet digital job-matching solutions remain inaccessible to most blue-collar workers due to three systemic barriers:

1. **Language Barrier**: Existing platforms operate primarily in English. Workers in Maharashtra, for instance, predominantly communicate in Marathi or Hindi.
2. **Literacy Barrier**: Many workers have limited text literacy, making conventional form-based profile creation impractical.
3. **Discovery Barrier**: Workers and employers operate within highly localized geographies (5–25 km radius). National-scale platforms fail to capture this hyper-local demand-supply dynamic.

2.2 Proposed Solution

LokRozgar AI provides:

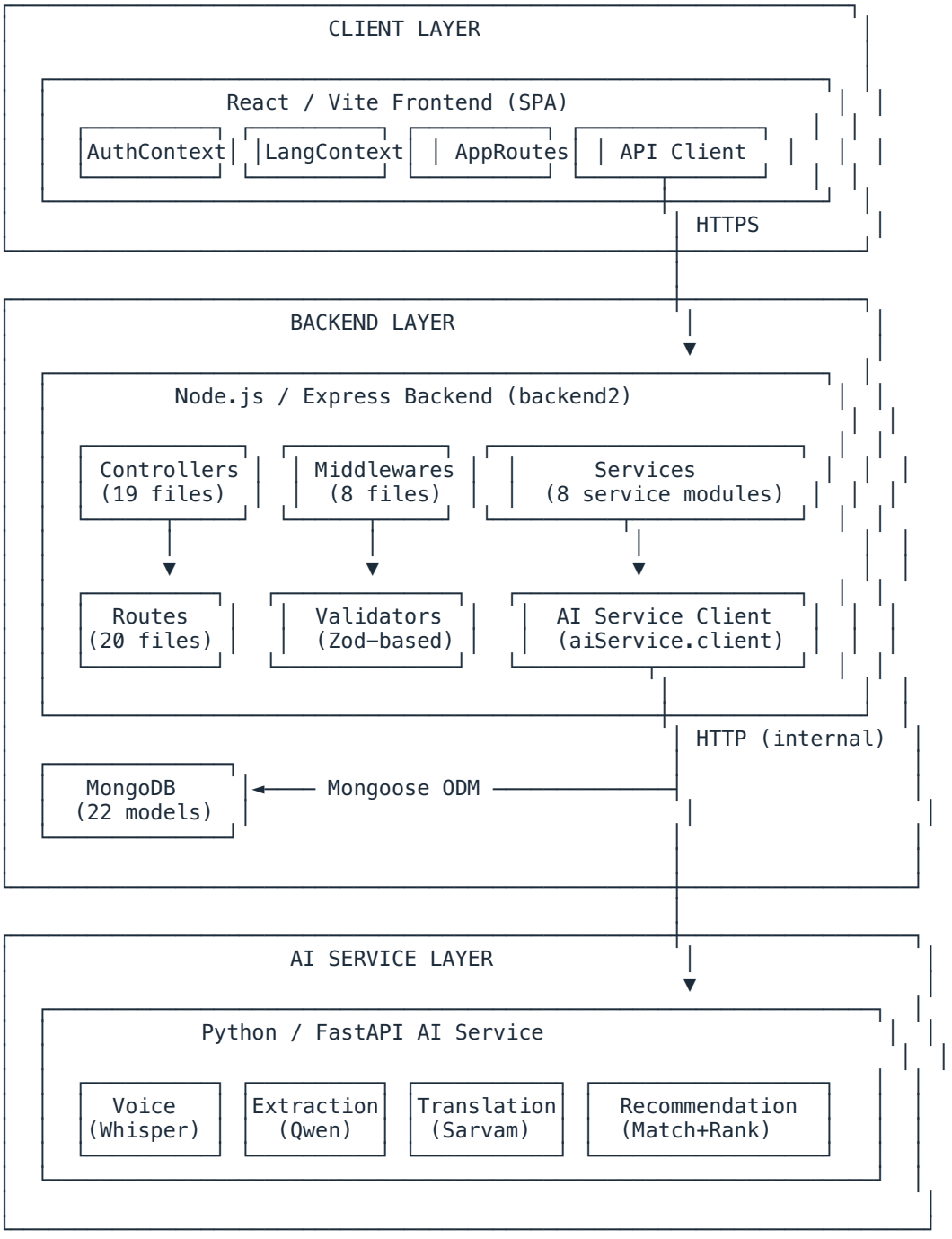
- **Voice-First Profile Creation**: Workers speak naturally in Marathi, Hindi, or English. The system transcribes, extracts structured profile fields (name, skills, experience, expected pay, availability, location), and populates their profile — no typing required.
- **Trilingual Interface**: Every user-facing string, job title, job description, skill name, and notification exists in Marathi (mr), Hindi (hi), and English (en). Language switching is instant and persisted per user.
- **Geospatial Job Matching**: MongoDB 2dsphere indices and the Haversine formula enable discovery within each worker's preferred travel radius.
- **AI-Powered Ranking**: A six-factor weighted scoring algorithm ranks candidates intelligently rather than by application timestamp.

2.3 Key Contributions

Contribution	Description
On-device voice pipeline	MLX Whisper + Qwen inference on Apple Silicon — no cloud dependency for core AI
Number grounding guard	Post-extraction validation ensures LLM-extracted numeric values actually appear in the transcript
Two-stage skill resolution	Deterministic alias matching followed by semantic LLM resolution against a canonical catalog
Six-factor weighted ranking	Skill, Location, Availability, Pay, Rating, Relevance — each computed independently, combined via configurable weights
Atomic capacity control	`findOneAndUpdate` prevents race-condition over-hiring when multiple employers accept simultaneously
Zero-audio-persistence policy	Voice recordings are processed as temporary streams and deleted immediately after extraction

3. System Architecture

3.1 High-Level Architecture



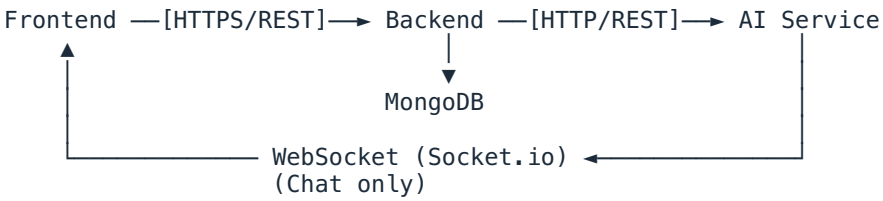
3.2 Service Boundaries & Ownership

Concern	Owner	Rationale
Authentication & JWT	Backend (Node)	Single point of token issuance/validation

Concern	Owner	Rationale
Authorization (RBAC)	Backend (Node)	Role checks bound to business rules
MongoDB (all reads/writes)	Backend (Node)	Single source of truth for data
Business rules (hiring, capacity)	Backend (Node)	Atomic operations require DB ownership
Job CRUD, Application lifecycle	Backend (Node)	State machine logic tightly coupled to DB
Voice transcription (STT)	AI Service (Python)	MLX Whisper / Sarvam AI integration
Profile field extraction	AI Service (Python)	Local LLM (Qwen) inference
Translation (Marathi/Hindi/English)	AI Service (Python)	Sarvam AI API integration
Matching, Ranking, Recommendation	AI Service (Python)	Algorithmic scoring pipeline
Skill semantic resolution	AI Service (Python)	LLM-backed catalog matching
Frontend UI & UX	Frontend (React)	Client-side rendering, no direct AI access

Critical Constraint: The frontend **never** communicates with the Python AI service directly. All AI operations flow through the Node backend, which acts as the sole orchestrator.

3.3 Communication Protocol



- **Frontend → Backend**: RESTful HTTPS with JWT bearer tokens; Axios interceptor handles 401 refresh.
- **Backend → AI Service**: Internal HTTP with shared API key (`AI_SERVICE_API_KEY`); never exposed externally.
- **Backend → Frontend**: WebSocket via Socket.io for real-time chat messages.

3.4 Deployment Topology

Component	Technology	Target
Frontend	Vite static build	Vercel / CDN
Backend	Node.js process	Render / Railway
AI Service	Uvicorn (ASGI)	Apple Silicon host (MLX)
Database	MongoDB	Atlas (M0 free tier)
File Storage	Cloudinary	Cloud CDN

Component	Technology	Target
SMS/OTP	MSG91 / Twilio	SaaS
Payments	Razorpay	SaaS

4. Technology Stack

4.1 Frontend

Component	Technology	Version
UI Framework	React	19.2.8
Build Tool	Vite	8.3.0
Routing	React Router DOM	7.18.4
HTTP Client	Axios	1.20.0
Icons	Lucide React	1.48.0
Linter	OxLint	1.81.0
Test Runner	Node.js native `--test`	—

4.2 Backend (Node/Express)

Component	Technology	Version
Runtime	Node.js	>= 20 (ESM)
Framework	Express	5.2.1
ODM	Mongoose	9.9.4
Authentication	JWT (jsonwebtoken)	9.0.3
Password Hashing	bcrypt	6.0.0
Validation	Zod	4.6.5
Rate Limiting	express-rate-limit	8.7.0
File Upload	Multer	2.3.0
Security Headers	Helmet	8.3.0
Real-time	Socket.io	4.8.3
Scheduling	node-cron	4.6.0
Payments	Razorpay SDK	2.9.8
SMS	MSG91 SDK, Twilio	1.0.7 / 6.1.1
Cloud Storage	AWS S3 SDK, Cloudinary	3.x / 2.11.0
Logging	Morgan	1.12.1

4.3 AI Service (Python/FastAPI)

Component	Technology	Version
Framework	FastAPI	0.141.1
ASGI Server	Uvicorn	0.54.0
Validation	Pydantic	2.13.5
STT (default)	MLX Whisper	0.4.3
STT (fallback)	OpenAI Whisper (PyTorch)	20250625
LLM Inference	MLX-LM (Qwen)	0.31.3
Metal Acceleration	MLX / MLX-Metal	0.32.3
Translation	Sarvam AI SDK	0.1.34
Deep Learning	PyTorch	2.14.0
NLP	Transformers (HuggingFace)	5.18.0
Tokenization	Tiktoken, SentencePiece	0.14.0 / 0.2.2
Testing	Pytest	9.1.1

4.4 Database

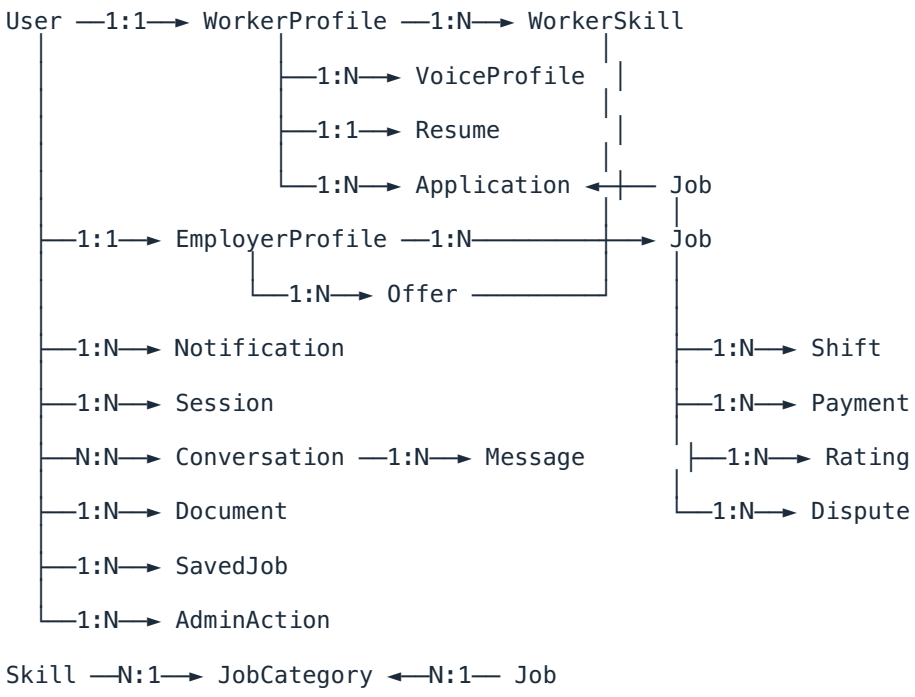
Property	Value
Engine	MongoDB 7.x
Hosting	MongoDB Atlas
ODM	Mongoose 9.9.4
Total Collections	22
Geospatial Index	2dsphere (Job, WorkerProfile, EmployerProfile)

4.5 External Services

Service	Provider	Purpose
Translation API	Sarvam AI	Marathi / Hindi / English text translation
Cloud STT (fallback)	Sarvam AI	Cloud-based speech-to-text
Object Storage	AWS S3 / Cloudinary	Profile photos, documents, audio
SMS / OTP	MSG91, Twilio	Phone verification
Payments	Razorpay	Online payment processing
Push Notifications	Firebase Admin	Mobile push notifications

5. Database Design

5.1 Entity-Relationship Model



5.2 Schema Definitions

5.2.1 User

The root identity entity. Supports **dual-role accounts** (a user can be both worker and employer simultaneously).

Field	Type	Constraints	Purpose
`phone`	String	**Required**, **Unique**, Trimmed	Primary identifier (phone-based auth)
`email`	String	Unique (partial filter: only when non-null)	Optional contact
`passwordHash`	String	Required	bcrypt hash
`roles`	[String]	`enum: ["worker", "employer", "admin"]`, min 1	Multi-role support
`preferredLanguage`	String	`enum: ["mr", "hi", "en"]`, default: `"en"`	UI language preference
`isVerified`	Boolean	default: `false`	Phone/identity verification status
`fraudScore`	Number	0-100, nullable	AI-computed risk score

Field	Type	Constraints	Purpose
`isFlagged`	Boolean	default: `false`	Admin moderation flag
`lastLoginAt`	Date	nullable	Session tracking

Indexes: `{ email: 1 }` (unique, partial: email exists)

5.2.2 WorkerProfile

Field	Type	Constraints	Purpose
`userId`	ObjectId -> User	**Required**, **Unique**	1:1 with User
`fullName`	String	Required, Trimmed	Display name
`bio`	`{ mr, hi, en }`	max 1000 chars each	Trilingual biography
`location`	GeoJSON Point	Required	GPS coordinates [lng, lat]
`availability`	String	`enum: ["available", "busy", "unavailable"]`	Current status
`expectedPay`	Number	>= 0	Daily wage expectation (INR)
`ratingAvg`	Number	0-5, default: 0	Aggregated star rating
`languages`	[String]	`enum: ["mr", "hi", "en"]`, default: `["mr"]`	Spoken languages
`preferredJobCategories`	[ObjectId -> JobCategory]	—	Category interests
`preferredWorkRadiusKm`	Number	>= 1, default: 20	Travel willingness
`experienceYears`	Number	>= 0, default: 0	Work experience

Indexes: `{ location: "2dsphere" }`

5.2.3 Job

Field	Type	Constraints	Purpose
`employerId`	ObjectId -> EmployerProfile	Required, Indexed	Job owner
`title`	`{ mr, hi, en }`	Nullable per language	Trilingual title
`description`	`{ mr, hi, en }`	Nullable per language	Trilingual description
`skillIds`	[ObjectId -> Skill]	—	Required skills
`location`	GeoJSON Point	Required	Job site coordinates

Field	Type	Constraints	Purpose
`payType`	String	`enum: ["daily", "monthly", "fixed"]`	Compensation type
`payAmount`	Number	>= 0, Required	Compensation amount (INR)
`requiredWorkers`	Number	>= 1, Required	Capacity
`status`	String	`enum: ["open", "assigned", "completed", "cancelled"]`	Lifecycle state
`startDate`	Date	Required	Job start date
`categoryId`	ObjectId -> JobCategory	Required, Indexed	Classification
`originalLanguage`	String	`enum: ["mr", "hi", "en"]`	Source language for translation
`experienceRequired`	Number	>= 0, default: 0	Minimum experience
`shiftType`	String	`enum: ["day", "night", "flexible"]`	Shift preference
`startTime` / `endTime`	String	—	Working hours
`applicationDeadline`	Date	—	Auto-expiry date

Indexes: `{ location: "2dsphere" }`, `{ status: 1, createdAt: -1 }`

5.2.4 Application

Field	Type	Constraints	Purpose
`jobId`	ObjectId -> Job	Required, Indexed	Target job
`workerId`	ObjectId -> WorkerProfile	Required, Indexed	Applicant
`status`	String	`enum: ["applied", "shortlisted", "accepted", "rejected", "completed", "withdrawn"]`	Lifecycle
`proposedPay`	Number	>= 0	Worker's counter-offer
`appliedAt`	Date	default: `Date.now`	Submission timestamp
`matchScore`	Number	0-100	AI-computed compatibility
`matchFactors`	Object	`{ skill, location, availability, pay, rating, relevance }`	Per-factor breakdown

Indexes: `{ jobId: 1, workerId: 1 }` (unique — one application per worker per job)

5.2.5 EmployerProfile

Field	Type	Constraints	Purpose
`userId`	ObjectId -> User	**Required**, **Unique**	1:1 with User
`businessName`	String	Required, Trimmed	Business display name
`businessType`	String	Required, Trimmed	Industry classification
`location`	GeoJSON Point	Required	Business site coordinates
`ratingAvg`	Number	0-5, default: 0	Aggregated employer rating
`bio`	String	Trimmed	Business description

Indexes: `{ location: "2dsphere" }`

5.2.6 Additional Collections Summary

Collection	Key Fields	Purpose
Skill	`name: { mr, hi, en }`, `categoryId`, `isActive`	Trilingual skill catalog
JobCategory	`name: { mr, hi, en }`, `description`, `isActive`	Job classification taxonomy
WorkerSkill	`workerId`, `skillId`, `proficiency`, `verified`	Worker-skill linkage with proficiency
VoiceProfile	`workerId`, `language`, `audioUrl`, `transcript`, `extractedFields`, `confidence`	Voice recordings per language
Resume	`workerId`, `headline`, `summary`, `experience[]`, `aiGenerated`, `sourceVoiceProfileId`	AI-generated or manual resumes
Offer	`jobId`, `employerId`, `workerId`, `proposedPay`, `status`, `expiresAt`	Direct job offers
Shift	`jobId`, `workerId`, `shiftDate`, `startTime`, `endTime`, `status`, `checkIn/Out`	Work shift scheduling
Payment	`jobId`, `payerId`, `payeeId`, `amount`, `currency`, `paymentMethod`, `gateway*`	Financial transactions
Rating	`jobId`, `fromUserId`, `toUserId`, `rating` (1-5), `review`	Mutual post-job feedback
Conversation	`participantIds[]`, `jobId`, `conversationType`	Chat threads (job or direct)
Message	`conversationId`, `senderId`, `messageType`, `text`, `audioUrl`, `isRead`	Chat messages
Notification	`userId`, `type`, `title: { mr, hi, en }`, `isRead`, `readAt`	Trilingual notifications
Dispute	`jobId`, `raisedByUserId`, `againstUserId`, `status`, `reason`, `resolvedByAdminId`	Conflict resolution

Collection	Key Fields	Purpose
Document	`userId`, `documentType`, `url`, `status`	Identity/certification documents
SavedJob	`userId`, `jobId`	Bookmarked jobs
Session	`userId`, `token`, `expiresAt`, `deviceInfo`	Active auth sessions
AdminAction	`adminId`, `targetUserId`, `action`, `reason`	Audit trail

5.3 Indexing Strategy

Index	Collection	Type	Purpose
`{ location: "2dsphere" }`	Job	Geospatial	Radius-based job discovery
`{ location: "2dsphere" }`	WorkerProfile	Geospatial	Nearby worker search
`{ location: "2dsphere" }`	EmployerProfile	Geospatial	Employer location queries
`{ status: 1, createdAt: -1 }`	Job	Compound	Open jobs sorted by recency
`{ jobId: 1, workerId: 1 }`	Application	Compound, Unique	Prevent duplicate applications
`{ jobId: 1, workerId: 1 }`	Offer	Compound, Unique	Prevent duplicate offers
`{ workerId: 1, language: 1 }`	VoiceProfile	Compound, Unique	One voice profile per language per worker
`{ jobId: 1, fromUserId: 1, toUserId: 1 }`	Rating	Compound, Unique	Prevent duplicate ratings
`{ userId: 1, isRead: 1, createdAt: 1 }`	Notification	Compound	Efficient notification queries
`{ conversationId: 1, createdAt: 1 }`	Message	Compound	Ordered message retrieval
`{ email: 1 }` (partial)	User	Unique, Partial	Email uniqueness only when provided

5.4 Multilingual Data Architecture

LokRozgar stores **all user-facing text** in three language variants at the database level:

```
// Pattern used across Job, Skill, JobCategory, Notification
title: {
  mr: String,  // Marathi
  hi: String,  // Hindi
  en: String,  // English
}
```

Design decisions:

- **No lazy translation**: All three variants are populated at creation time (via Sarvam AI for jobs, via manual entry for catalog data).
- **Original language preserved**: Every Job records its `originalLanguage`, enabling the UI to indicate machine-translated content.
- **Frontend resolution**: The `LanguageContext` selects the appropriate variant based on the user's current language preference.
- **Fallback chain**: `current_language -> en -> key` ensures no blank content even if a translation is missing.

---

6. Authentication & Authorization

6.1 Authentication Flow



1. **Registration** creates a `User` document with bcrypt-hashed password, plus role-specific profiles (`WorkerProfile` and/or `EmployerProfile`).
2. **Login** validates credentials, issues JWT access + refresh token pair.
3. **Bootstrap**: On app load, the frontend calls `GET /users/me` with the stored access token to hydrate `AuthContext`.

6.2 Token Management

Token	Lifetime	Storage	Purpose
Access Token	Short-lived (15 min)	`localStorage`	Request authorization
Refresh Token	Long-lived (7 days)	`localStorage`	Silent token renewal

The Axios interceptor in `client.js` intercepts 401 responses, attempts a refresh, replays the failed request, and fires an `AUTH_EXPIRED_EVENT` if the refresh also fails — triggering `AuthContext.clearSession()`.

6.3 Role-Based Access Control

Three roles exist: `worker`, `employer`, `admin`. A single user may hold multiple roles (e.g., both `worker` and `employer`).

Middleware	File	Logic
`auth.middleware.js`	JWT verification	Validates token, attaches `req.user`

Middleware	File	Logic
`role.middleware.js`	Role intersection check	Rejects if no overlap with `allowedRoles`
`profile.middleware.js`	Profile existence check	Ensures `WorkerProfile`/`EmployerProfile` exists

Frontend route guards mirror backend RBAC via `ProtectedRoute`:

```
<ProtectedRoute allowedRoles={[USER_ROLES.WORKER]}>
  <WorkerDashboardPage />
</ProtectedRoute>
```

6.4 Rate Limiting Strategy

Implemented in `rateLimiter.middleware.js` using `express-rate-limit`:

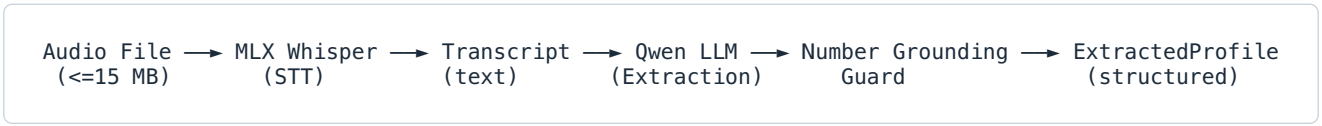
Mode	Key	Window	Max Requests	Purpose
Production	IP	15 min	100	Brute-force protection
Development	Phone + IP	15 min	1000	Per-account limiting without blocking shared IPs
Auth endpoints	IP	15 min	20	Stricter limit on login/register

7. AI & Machine Learning Pipeline

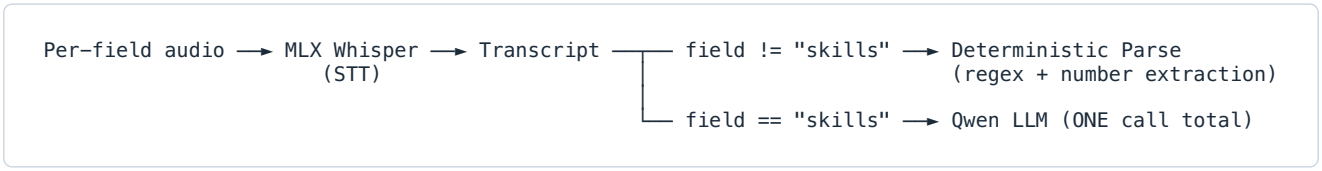
7.1 Voice Processing Pipeline

The voice pipeline supports two flows: **Quick** (single recording, full profile extraction) and **Guided** (step-by-step, one field per recording).

Quick Flow:



Guided Flow:



STT Provider Architecture (Strategy Pattern):

Provider	Environment Variable	Technology	Use Case
`mlx_whisper` (default)	`STT_PROVIDER=mlx_whisper`	MLX Whisper (Metal/ANE)	Production on Apple Silicon
`whisper`	`STT_PROVIDER=whisper`	OpenAI Whisper (PyTorch)	Non-Apple hardware fallback
`sarvam`	`STT_PROVIDER=sarvam`	Sarvam AI Cloud API	Cloud deployment

All three implement the `STTProvider` interface:

```
async def transcribe(audio_bytes, filename, content_type, language) -> (transcript, confidence)
```

Provider selection is fresh on every call (via `STT_PROVIDER` env var), allowing hot-switching during development without process restart.

7.2 Profile Extraction Engine

Provider Architecture (Strategy Pattern):

Provider	Env Variable	Technology	Default
`local_llm`	`EXTRACTION_PROVIDER=local_llm`	MLX-LM (Qwen on Apple Silicon)	Yes
`sarvam`	`EXTRACTION_PROVIDER=sarvam`	Sarvam AI API	Fallback

Output Schema (`ExtractedProfile`):

```
class ExtractedProfile(BaseModel):
    full_name:      Optional[str]        # "Ramesh Patil"
    experience_years: Optional[float]    # 6.0
    expected_pay:   Optional[float]      # 800.0
    skills:         Optional[List[str]]  # ["electrician", "wiring"]
    city:           Optional[str]        # "Sangli"
    bio:            Optional[str]        # Free-text biography
    availability:   Optional[str]        # "available" | "busy" | "unavailable"
    languages:      Optional[List[str]]  # ["mr", "hi"]
    preferred_job_categories: Optional[List[str]]  # ["construction"]
```

Guided Flow Field Extraction (`field_extraction.py`):

For all fields except `skills`, extraction is **deterministic** (no LLM call):

Field	Extraction Method	LLM Required
`full_name`	First non-empty line of transcript	No
`experience_years`	Regex number extraction with Indian numeral word support	No
`expected_pay`	Regex number extraction with currency word support	No
`city`	Fuzzy matching against known city list	No
`availability`	Keyword matching (mr/hi/en variants of available/busy/unavailable)	No
`skills`	Full LLM extraction, only `skills` list taken	**Yes**

Only the `skills` field triggers the Qwen LLM — making guided sessions use exactly **one** LLM call total (~11-17s vs ~3s for transcription alone).

7.3 Number Grounding Guard

Problem: The LLM occasionally hallucinates numeric values. A transcript saying "eight hundred rupees" (said once) might produce `expectedPay: 10000` — a fabricated number that never appeared in the speech.

Solution: `number_grounding.py` post-validates extracted numeric fields:

```
def _drop_ungrounded_numbers(profile, transcript):
    for field in ("experience_years", "expected_pay"):
        value = getattr(profile, field)
        if value is not None and not is_grounded(value, transcript):
            replacements[field] = None  # Drop — let the worker fill it manually
    return profile.model_copy(update=replacements)
```

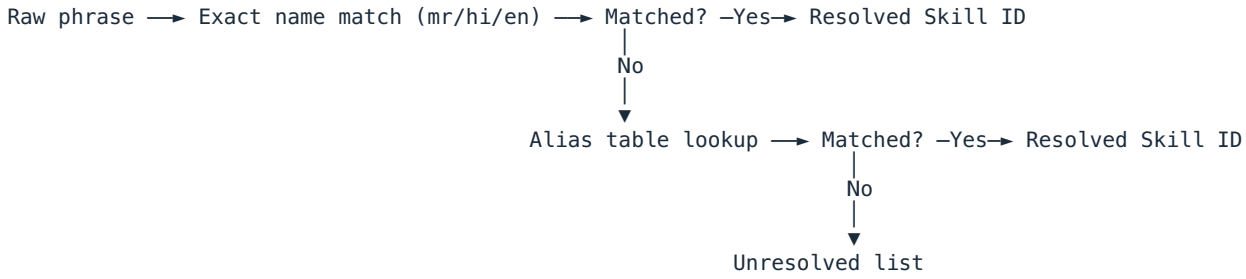
The `is_grounded(value, transcript)` function checks whether the numeric value appears in the transcript in **any** form: digit form, Indian number words in Marathi, Hindi, or English. This is a **conservative** guard: it only ever clears a value — it never invents, rounds, or repairs one.

Rationale: An empty field the worker fills in manually is recoverable. A confident wrong number silently saved to their profile is not.

7.4 Skill Resolution Pipeline

A two-stage pipeline resolves free-form skill phrases (from voice extraction) to canonical catalog entries:

Stage 1: Deterministic Resolution (Node.js — `workerSkill.controller.js`)



Stage 2: Semantic Resolution (AI Service — `POST /skills/resolve`)

Only unresolved phrases from Stage 1 are sent. The Qwen LLM selects the best match from the provided catalog (never invents a skill outside the catalog).

```
POST /skills/resolve
{ rawSkills: ["electrician work"], catalog: [{id, name: {mr,hi,en}}] }

Response: { results: [{ raw: "electrician work", skillId: "...", confidence: 0.92 }] }
```

Key Design: Node owns the final acceptance decision (confidence >= threshold -> accept). The AI service only reports what the LLM selected; it cannot persist a `WorkerSkill` document.

7.5 Translation Service

Endpoint: `POST /translation/translate`

Provider: Sarvam AI translation model

Algorithm:

1. Accept source text and source language code (`mr`, `hi`, or `en`).
2. Identify the two target languages (the other two of mr/hi/en).
3. Run **two concurrent** translation calls via `asyncio.gather` (source -> target_1, source -> target_2).
4. Never chain translations (no hi->en->mr). Each translation runs independently from the source to prevent error accumulation.
5. Return `{ mr, hi, en }` — the source language's field contains the original text verbatim.

Resilience: The AI service raises on failure. The backend's `translation.service.js` wrapper implements the "never block the caller" policy — if translation fails, the job is still created with only the original language populated.

8. Matching & Ranking System

8.1 Architecture Overview



8.2 Six-Factor Matching Algorithm

Each factor is independently computed, returns a score on a **0-100 scale** (or `None` if computation is impossible), and has a dedicated feature-adapter layer that extracts inputs from the domain models.

Factor 1: Skill Match

```
score = |worker_skills intersect job_skills| / |job_skills| * 100
```

- If `job_skills` is empty: score = 0 (no requirements to match against).
- Returns matched and missing skill ID lists for transparency.
- Comparison uses canonical skill IDs (ObjectId strings), not free-text names.

Factor 2: Location Match (Haversine)

Uses the **Haversine formula** for great-circle distance calculation:

```
distance_km = haversine(worker_location, job_location)

if distance_km < preferred_radius_km:
    score = (1 - distance_km / preferred_radius_km) * 100
else:
    score = 0
```

Edge Case	Behavior
Missing location (either side)	`score = None` (excluded from ranking, not treated as 0)
`preferred_radius_km <= 0`	`score = 0`
`distance_km = 0`	`score = 100` (co-located)

The Haversine implementation uses GeoJSON coordinate order `[longitude, latitude]` and Earth radius of 6371.0 km.

Factor 3: Availability Match

```
if worker.availability == "available" AND job.status == "open":
    score = 100
elif worker.availability == "busy" AND job.status == "open":
    score = 50
else:
    score = 0
```

Binary/ternary logic — no continuous gradient. An "unavailable" worker still enters the scoring pipeline (they are not filtered out), but receives score 0 for this factor.

Factor 4: Pay Match

```
score = None  (v1 limitation)
```

In the current implementation, pay matching always returns `None` because `WorkerProfile.expectedPay` has no declared unit (daily/monthly/fixed), making comparison with `Job.payAmount` + `Job.payType` unsafe. This is a known v1 limitation — the factor is excluded from ranking (not treated as 0), so it has no negative impact on overall scores.

Factor 5: Rating Match

```
if worker.ratingAvg == 0:
    score = None  (no ratings on record — cold start)
else:
    score = (worker.ratingAvg / 5.0) * 100
```

A `None` score means the worker has never been rated — it is "no data available," not "poor rating."

Factor 6: Relevance Match

Combines three sub-signals via configurable weights:

Sub-signal	Logic	Default Weight
Category	`job.categoryId in worker.preferredJobCategories`	1.0
Language	Any worker language matches job's preferred languages (or original language)	1.0
Experience	`worker.experienceYears >= job.experienceRequired`	1.0

```
score = sum(matched_signal_weight) / sum(applicable_signal_weight) * 100
```

If a worker has no `preferredJobCategories`, the category signal is "not applicable" and excluded from both numerator and denominator (not counted against them).

8.3 Ranking Engine

The ranking engine (`rank.py`) combines the six factor scores into a single `overall_score` using a **weighted average**:

```
overall_score = sum(factor_score_i * weight_i) / sum(weight_i)
    where i iterates over factors with non-None scores only
```

Configuration (`RankingWeights`):

```
class RankingWeights(BaseModel):
    skill:        float = 1.0  # >= 0
    location:     float = 1.0
    availability: float = 1.0
    pay:          float = 1.0
    rating:       float = 1.0
    relevance:    float = 1.0
```

Key properties:

- Equal weighting by default (configurable per-request via the API).
- `None` factors are excluded from **both** numerator and denominator — they are never treated as 0.
- All-zero weights are rejected at construction time (Pydantic `model_validator`).
- `overall_score = None` when no factors are available (pair is "unrankable," not "worst possible").
- Output is clamped to [0, 100] as a safety guard, though mathematically redundant given non-negative weights and 0-100 inputs.

8.4 Recommendation Pipeline

Two entry points, symmetric in structure:

Endpoint	Direction	Anchor	Candidates
`recommend_jobs_for_worker()`	Worker -> Jobs	One worker	All open/partially-assigned jobs
`recommend_workers_for_job()`	Job -> Workers	One job	All worker profiles

Algorithm:

```
1. Filter candidates (eligibility gate — status-based only, from candidates.py)
2. For each candidate: score_worker_job_pair() -> RankingResult
3. Discard pairs with overall_score = None (unrankable)
4. Sort by overall_score DESC (ties broken by ID for determinism)
5. Truncate to top_k (default: 10)
6. Attach human-readable explanation per match via explain_ranking_result()
7. Return RecommendationResponse
```

Candidate eligibility (`candidates.py`):

- `status == "open"` -> eligible
- `status == "partially_assigned"` AND `filledWorkers < requiredWorkers` -> eligible
- All other statuses -> not eligible

Skill, location, availability, pay, rating, experience, and language are deliberately **NOT** filtered at this stage — they are already handled by the scoring pipeline, and pre-filtering would duplicate that system.

8.5 Explanation Generator

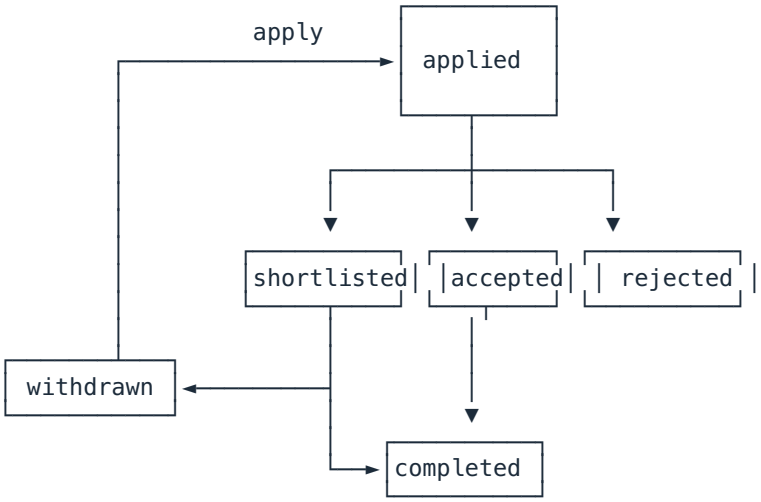
Each recommendation includes a structured, deterministic explanation:

```
{
  "overall_score": 72.5,
  "factors": [
    { "factor": "skill", "available": true, "score": 100.0,
      "summary": "Skill compatibility scored 100.0 out of 100." },
    { "factor": "location", "available": true, "score": 85.3,
      "summary": "Location proximity scored 85.3 out of 100." },
    { "factor": "availability", "available": true, "score": 100.0,
      "summary": "Availability scored 100.0 out of 100." },
    { "factor": "pay", "available": false, "score": null,
      "summary": "Pay compatibility is unavailable: the worker's expected pay has no declared unit..." },
    { "factor": "rating", "available": false, "score": null,
      "summary": "Worker rating is unavailable: the worker has not received any ratings yet..." },
    { "factor": "relevance", "available": true, "score": 66.7,
      "summary": "Profile/job relevance scored 66.7 out of 100." }
  ],
  "summary": "Overall match score: 72.5 out of 100, based on 4 of 6 factors (pay, rating unavailable)."
}
```

Factor order is always fixed: skill, location, availability, pay, rating, relevance — regardless of how `MatchFactors` was built internally.

9. Application & Hiring Flow

9.1 Application Lifecycle



State transitions (enforced by `application.controller.js`):

From	To	Actor	Condition
—	`applied`	Worker	Job is open, no existing application
`applied`	`shortlisted`	Employer	—
`applied` / `shortlisted`	`accepted`	Employer	`filledWorkers < requiredWorkers`
`applied` / `shortlisted`	`rejected`	Employer	—
`applied` / `shortlisted`	`withdrawn`	Worker	—
`accepted`	`completed`	Employer	Job marked complete

At-application AI scoring: When a worker applies for a job, the backend calls `POST /recommendations/score-pair` on the AI service, receiving a full `RankingResult` + `MatchExplanation`. The `matchScore` and `matchFactors` are persisted on the Application document, enabling:

- Sorted applicant views for employers (ranked by match score)
- Transparency into why a worker was scored a certain way

9.2 Offer Management

Employers can proactively send offers to workers (bypassing the application flow):

Field	Type	Description
`jobId`	ObjectId	Target job

Field	Type	Description
`employerId`	ObjectId	Offer sender
`workerId`	ObjectId	Offer recipient
`proposedPay`	Number	Employer's pay offer (INR)
`status`	String	`sent` -> `accepted` / `rejected` / `expired`
`expiresAt`	Date	Auto-expiry deadline

Constraint: `{ jobId, workerId }` unique index — one offer per worker per job.

9.3 Capacity Control & Atomic Hiring

Problem: Multiple employers (or concurrent requests) processing accept decisions simultaneously can cause over-hiring — accepting more workers than `requiredWorkers`.

Solution: Mongoose `findOneAndUpdate` with atomic condition check and increment:

```
// Simplified pseudocode from application.controller.js
const job = await Job.findOneAndUpdate(
  {
    _id: jobId,
    filledWorkers: { $lt: requiredWorkers }  // Atomic guard
  },
  { $inc: { filledWorkers: 1 } },
  { new: true }
);

if (!job) {
  // Capacity full — reject the accept attempt
  throw new ApiError(409, "All positions are already filled");
}
```

This guarantees:

- **No race condition**: The `$lt` condition and `$inc` are atomic at the MongoDB engine level.
- The transition to `assigned` status happens only when `filledWorkers >= requiredWorkers`.
- `partially_assigned` is set when `0 < filledWorkers < requiredWorkers`.

9.4 Shift Management

After hiring, individual work shifts are scheduled:

Field	Description
`shiftDate`	Specific work day
`startTime` / `endTime`	Work hours (string format, e.g., "09:00", "18:00")
`status`	`scheduled` -> `completed` / `missed` / `cancelled`
`checkInAt` / `checkOutAt`	Actual attendance timestamps

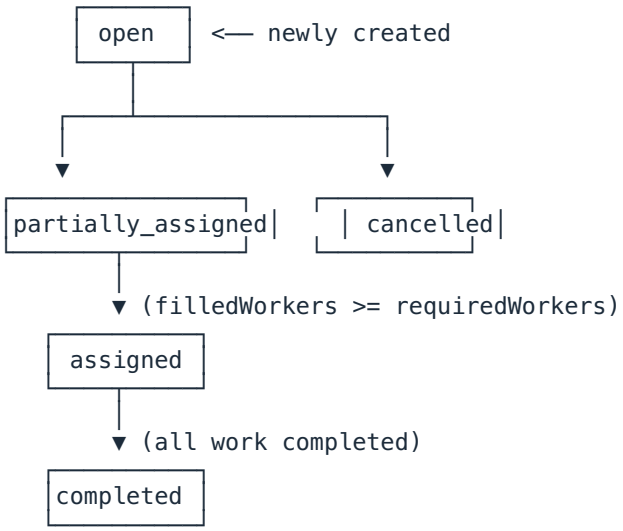
10. Job Posting & Management

10.1 Job Creation with Auto-Translation

```
Employer submits job in originalLanguage (e.g., Marathi)
      |
      ▼
Backend receives { title, description, originalLanguage: "mr" }
      |
      |──► Immediately stores the original-language fields
      |
      |──► POST /translation/translate { text: title, source_lang: "mr" }
      |    Response: { mr: <original>, hi: <translated>, en: <translated> }
      |
      |──► POST /translation/translate { text: description, source_lang: "mr" }
      |    Response: { mr: <original>, hi: <translated>, en: <translated> }
      |
      ▼
Backend populates all three language fields and saves:
  title: { mr: "...", hi: "...", en: "..." }
  description: { mr: "...", hi: "...", en: "..." }
```

Resilience: If translation fails (Sarvam API down, network error), the job is still created with only the original language filled. The other two fields remain `null`. The frontend gracefully falls back to displaying whatever variant is available.

10.2 Job Lifecycle State Machine



Cron job (`node-cron`): Expired jobs (past `applicationDeadline`) are automatically transitioned to `expired` status by a scheduled task.

10.3 Geospatial Querying

MongoDB's `2dsphere` index enables efficient spatial queries:

```
// Find jobs within radius (meters)
Job.find({
  location: {
    $near: {
      $geometry: {
        type: "Point",
        coordinates: [workerLng, workerLat]
      },
      $maxDistance: radiusKm * 1000
    }
  },
  status: "open"
});
```

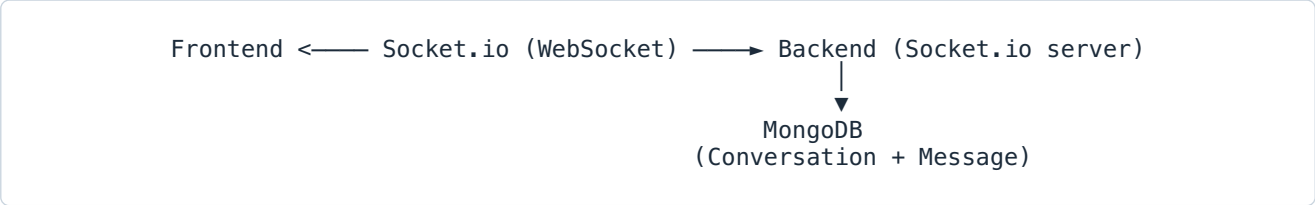
Both `Job` and `WorkerProfile` collections carry `2dsphere` indexes, enabling bidirectional spatial discovery (jobs near a worker, and workers near a job).

---

11. Communication System

11.1 Real-Time Chat (Socket.io)

Architecture:



Conversation Types:

- `job`: Linked to a specific job (employer-worker discussion about a posting)
- `direct`: Free-form messaging between any two authenticated users

Message Types: `text`, `voice`, `image`, `system`

Authorization: Per-conversation participant check (not per-role). A dual-role user can be the worker in one conversation and the employer in another. The server verifies participant membership on every message send/receive operation.

Message indexing: `{ conversationId: 1, createdAt: 1 }` ensures messages are retrieved in chronological order efficiently.

11.2 Notification System

Trilingual notifications (`title: { mr, hi, en }`) are created on key events:

Event	Notification Type	Recipient
New application received	`application_received`	Employer
Application accepted	`application_accepted`	Worker
Application rejected	`application_rejected`	Worker
New offer received	`offer_received`	Worker
Job assigned	`job_assigned`	Worker
Rating received	`rating_received`	Worker / Employer

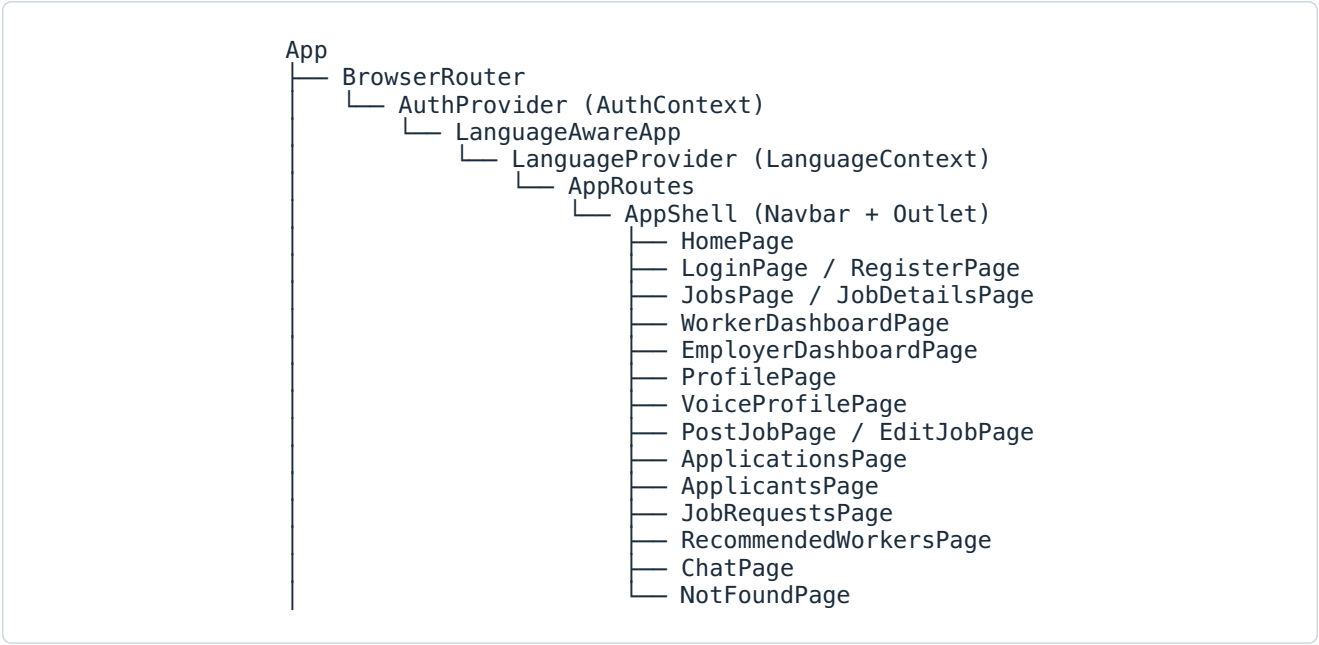
Notifications are stored in MongoDB and delivered via:

1. **In-app polling**: `GET /api/v1/notifications` (with read/unread filtering)
2. **Push notification**: Firebase Admin SDK (for mobile clients)

Notification service (`notification.service.js`): Creates trilingual notification documents and optionally dispatches Firebase push notifications.

12. Frontend Architecture

12.1 Component Hierarchy



12.2 Routing & Guards

Route	Page	Access	Role Guard
`/`	HomePage	Public	—
`/login`	LoginPage	Public	—
`/register`	RegisterPage	Public	—
`/jobs`	JobsPage	Public	—
`/jobs/:id`	JobDetailsPage	Public	—
`/worker-dashboard`	WorkerDashboardPage	Authenticated	Worker
`/employer-dashboard`	EmployerDashboardPage	Authenticated	Employer
`/profile`	ProfilePage	Authenticated	Any role
`/voice-profile`	VoiceProfilePage	Authenticated	Worker
`/applications`	ApplicationsPage	Authenticated	Worker
`/job-requests`	JobRequestsPage	Authenticated	Worker
`/post-job`	PostJobPage	Authenticated	Employer
`/jobs/:id/edit`	EditJobPage	Authenticated	Employer
`/applicants/:jobId`	ApplicantsPage	Authenticated	Employer
`/jobs/:jobId/recommended-workers`	RecommendedWorkersPage	Authenticated	Employer

Route	Page	Access	Role Guard
`/messages`	ChatPage	Authenticated	Any role
`*`	NotFoundPage	Public	—

12.3 State Management

Two React Context providers manage global state:

AuthContext

State	Type	Description
`user`	Object or null	Current user document
`workerProfile`	Object or null	Worker profile (if worker role)
`employerProfile`	Object or null	Employer profile (if employer role)
`isAuthenticated`	Boolean	Derived from `user` presence
`isBootstrapped`	Boolean	True after initial auth check completes

Methods: `login()`, `register()`, `logout()`, `refreshBootstrap()`

LanguageContext

State	Type	Description
`language`	`"en"` or `"hi"` or `"mr"`	Current UI language
`t(key, vars?)`	Function	Translation lookup with variable interpolation

Resolution chain: `current_language_dict -> English dict -> raw key`

Persistence: Language preference is stored in `localStorage` and synced to the server (`PUT /api/v1/users/language`) on change. The sync is best-effort — it never blocks the UI language switch.

12.4 Internationalization (i18n)

Three static dictionaries (`en.js`, `hi.js`, `mr.js`) provide all UI strings:

```
// Example: mr.js (Marathi)
export default {
  welcome: "स्वागत आहे",
  findJob: "नोकरी शोधा",
  postJob: "नोकरी द्या",
  myProfile: "माझी माहिती",
  skills: "कौशल्ये",
  // ... 200+ keys
};
```

Variable interpolation: `t("greeting", { name: "Ramesh" })` resolves `"नमस्कार, {{name}}"` to `"नमस्कार, Ramesh"`.

Design: The `LanguageProvider` wraps the entire app below `AuthProvider`. When a user logs in and their `preferredLanguage` is known, it auto-adopts that preference (unless the user has already explicitly stored a local choice).

12.5 Voice Profile UI Workflow

The `VoiceProfilePage` supports two flows:

Quick Flow:

1. Worker taps the record button and speaks freely in their language.
2. A single audio file is uploaded to `POST /voice/extract` (via the backend proxy).
3. The AI service returns a full `ExtractedProfile` with all detected fields.
4. The UI displays the transcript and each extracted field for review.
5. The worker confirms or manually edits any field before saving.

Guided Flow (Step-by-Step Wizard):

1. The wizard walks the worker through each field: Name -> Experience -> Expected Pay -> City -> Skills -> Availability.
2. One short recording per step is uploaded to `POST /voice/extract-field` with the target `field` parameter.
3. Deterministic extraction handles all fields except "skills" (no LLM latency).
4. The skills step triggers the one LLM call of the entire session.
5. Each step shows the raw transcript alongside the extracted value for immediate confirmation.

---

13. API Reference

13.1 Backend REST API

All routes are prefixed with `/api/v1`.

Authentication (`/api/v1/auth`)

Method	Route	Description	Auth
POST	`/register`	Create user + profile(s)	Public
POST	`/login`	Authenticate, return JWT pair	Public
POST	`/refresh-token`	Refresh access token	Refresh token
POST	`/logout`	Invalidate session	Bearer

Users (`/api/v1/users`)

Method	Route	Description	Auth
GET	`/me`	Current user + all profiles	Bearer
PUT	`/language`	Update preferred language	Bearer

Jobs (`/api/v1/jobs`)

Method	Route	Description	Auth
GET	`/`	List/search/filter jobs (with geo, category, skill filters)	Public
POST	`/`	Create job (with auto-translation)	Employer
GET	`/:id`	Job details with populated employer + skills	Public
PUT	`/:id`	Update job fields	Employer (owner)
DELETE	`/:id`	Cancel/delete job	Employer (owner)
GET	`/:id/recommended-workers`	AI-recommended workers for this job	Employer

Applications (`/api/v1/applications`)

Method	Route	Description	Auth
POST	`/`	Apply to job (triggers AI scoring)	Worker
GET	`/my`	Worker's own applications	Worker
GET	`/job/:jobId`	All applicants for a job (sorted by match score)	Employer
PUT	`/:id/status`	Accept / reject / shortlist an application	Employer
PUT	`/:id/withdraw`	Withdraw an application	Worker

Worker Profile (`/api/v1/worker-profile`)

Method	Route	Description	Auth
GET	`/`	Get current worker profile	Worker
PUT	`/`	Update worker profile fields	Worker

Employer Profile (`/api/v1/employer-profile`)

Method	Route	Description	Auth
GET	`/`	Get current employer profile	Employer
PUT	`/`	Update employer profile fields	Employer

Voice Profile (`/api/v1/voice-profile`)

Method	Route	Description	Auth
POST	`/extract`	Full voice extraction (quick flow)	Worker
POST	`/extract-field`	Per-field extraction (guided flow)	Worker
GET	`/`	Get existing voice profiles	Worker

Chat (`/api/v1/chat`)

Method	Route	Description	Auth
GET	`/conversations`	List all user's conversations	Bearer
POST	`/conversations`	Create or get existing conversation	Bearer
GET	`/conversations/:id/messages`	Get messages for a conversation	Bearer
POST	`/conversations/:id/messages`	Send a message	Bearer

Additional Endpoints

Resource	Prefix	Key Operations
Ratings	`/api/v1/ratings`	Submit rating, get ratings for user/job
Payments	`/api/v1/payments`	Create payment, verify Razorpay, list payments
Notifications	`/api/v1/notifications`	List notifications, mark as read
Categories & Skills	`/api/v1/catalog`	List categories, list skills by category
Worker Skills	`/api/v1/worker-skills`	Add skill, remove skill, list worker's skills
Shifts	`/api/v1/shifts`	Create shift, update status, check in/out
Disputes	`/api/v1/disputes`	Raise dispute, admin resolve
Admin	`/api/v1/admin`	User management, flag/unflag, analytics

Resource	Prefix	Key Operations
Documents	`/api/v1/documents`	Upload document, admin verify
Saved Jobs	`/api/v1/saved-jobs`	Save job, unsave, list saved
Resumes	`/api/v1/resumes`	Generate AI resume, get resume

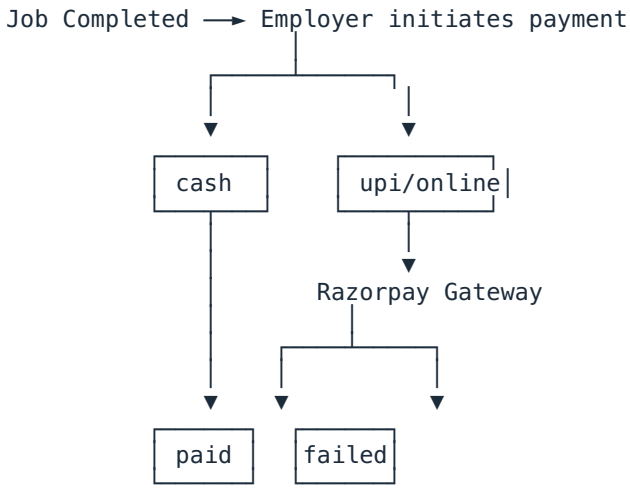
13.2 AI Service API

All AI service endpoints require `X-API-Key` header authentication (shared secret between Node backend and AI service).

Method	Route	Request Body	Response	Purpose
GET	`/health`	—	`{ status: "ok" }`	Liveness check
POST	`/voice/extract`	`multipart/form-data` (audio file, language)	`VoiceExtractionResponse`	Full profile extraction from audio
POST	`/voice/extract-field`	`multipart/form-data` (audio file, language, field)	`VoiceFieldResponse`	Single-field extraction from audio
POST	`/translation/translate`	`{ text, source_lang }`	`{ mr, hi, en }`	Trilingual text translation
POST	`/skills/resolve`	`{ rawSkills[], catalog[] }`	`{ results: [SkillResolutionResult] }`	Semantic skill resolution
POST	`/recommendations/jobs-for-worker`	`JobsForWorkerRequest`	`RecommendationResponse`	Top-K job recommendations
POST	`/recommendations/workers-for-job`	`WorkersForJobRequest`	`RecommendationResponse`	Top-K worker recommendations
POST	`/recommendations/score-pair`	`ScorePairRequest`	`MatchWithExplanation`	Single worker-job pair scoring

14. Payment & Dispute Resolution

Payment Flow



Payment Schema:

- **Methods**: `cash`, `upi`, `online`
- **Gateways**: `razorpay`, `other`
- **Statuses**: `pending` -> `processing` -> `paid` / `failed` / `refunded`
- **Currency**: INR (default)
- Gateway-specific fields: `gatewayTransactionId`, `gatewayOrderId`, `gatewayResponse` (raw response storage for audit)

Dispute Resolution

Status	Description
`open`	Dispute raised by a user
`in_review`	Admin currently reviewing
`resolved`	Decision made by admin
`rejected`	Dispute dismissed

Disputes are linked to a specific `jobId` and track both the raising user (`raisedByUserId`) and the accused user (`againstUserId`), plus an optional admin resolver (`resolvedByAdminId`).

15. Security Architecture

Layer	Mechanism	Implementation
Transport	HTTPS (TLS)	Deployment-level (Render/Vercel)
Headers	Helmet.js	XSS, CSRF, clickjacking, MIME-sniffing protection
Authentication	JWT (HS256)	`jsonwebtoken` library
Password Storage	bcrypt (cost factor 10+)	`bcrypt` library
Input Validation	Zod schemas	Request body validation per endpoint
Rate Limiting	express-rate-limit	IP-based (prod) / Phone+IP (dev)
CORS	Origin whitelist	`cors()` with configurable `CORS_ORIGIN`
File Upload	Multer with size/MIME limits	15 MB audio, 5 MB images
Internal API Auth	Shared secret key	`X-API-Key` between Node and AI service
Audio Privacy	Zero-persistence policy	Temp files deleted after extraction
Injection Prevention	Mongoose ODM	Parameterized queries, no raw string interpolation
Error Masking	ApiError class	Stack traces only in development mode
CastError Handling	Global error handler	Malformed ObjectIDs return 400 (not 500)

Zero-Audio-Persistence Policy: Voice recordings are received as temporary multipart uploads, streamed to the AI service for processing, and deleted immediately after. No audio file is persisted beyond the request lifecycle. Only the extracted transcript and structured fields are stored.

16. Testing Strategy

Layer	Framework	Test Count	Coverage Focus
Frontend	Node.js native `--test`	94	Component rendering, API integration, routing
Backend	Node.js native test runner	459	Controllers, middlewares, validators, services
AI Service	Pytest	526	Matchers, ranking, recommendation, extraction, voice
Total		**1,079**	

AI Service Test Architecture:

- **Unit tests**: Each of the six matcher functions tested in isolation with comprehensive edge cases (None inputs, zero values, boundary conditions).
- **Integration tests**: Full `score_worker_job_pair()` pipeline tested with realistic worker/job data.
- **Property tests**: Ranking weight validation, score boundary checks, deterministic factor ordering.
- **Regression tests**: Number grounding guard against known LLM hallucination patterns, field extraction edge cases.
- **Provider tests**: Both STT providers (MLX Whisper, Sarvam) and extraction providers (local LLM, Sarvam) tested independently.

17. Performance Considerations

Concern	Solution
LLM Inference Concurrency	Semaphore-protected inference prevents unified memory contention on Apple Silicon
MLX Whisper Latency	~3s per transcription on M1/M2 with Metal/ANE acceleration
Qwen LLM Latency	~11-17s per extraction call; guided flow limits to 1 call per session
Database Query Efficiency	2dsphere geospatial indexes, compound indexes for all frequent query patterns
Translation Concurrency	Two target languages translated in parallel via `asyncio.gather`
Frontend Bundle	Vite tree-shaking, code splitting per route, optimized production build
API Response Size	Pagination on list endpoints, `top_k` truncation on recommendations
Token Refresh	Silent refresh via Axios interceptor — no user-visible re-authentication
CORS vs Rate Limiter	CORS registered before rate limiter to ensure 429 responses carry correct CORS headers
MongoDB Connection	Single Mongoose connection pool, connection established before server accepts requests

18. Future Roadmap

Feature	Priority	Description
Pay unit declaration	High	Add `expectedPayType` to WorkerProfile to enable Pay Match scoring
Aadhaar KYC integration	High	Government identity verification via DigiLocker/Aadhaar eKYC
Offline mode	Medium	Job browsing cache with sync-on-reconnect
Push notifications (mobile)	Medium	Firebase FCM integration for mobile app users
Admin analytics dashboard	Medium	Platform metrics, fraud detection, engagement analytics
Skill badge verification	Medium	Verified badges via completed jobs + employer ratings
UPI direct payment	Low	Razorpay UPI intent flow for in-app payments
ASEEM portal integration	Low	Government employment exchange API (Ministry of Skill Development)
Worker portfolio (photos)	Low	Visual work samples for skilled workers (e.g., carpentry, electrical work)
Regional language expansion	Low	Kannada, Telugu, Tamil, Gujarati support
WebSocket notifications	Low	Replace polling with real-time notification delivery

19. Glossary

Term	Definition
Blue-collar worker	Manual or trade-skilled labourer (e.g., electrician, plumber, driver, carpenter, mason)
STT	Speech-to-Text — converting audio speech into textual transcript
MLX	Apple's Machine Learning framework optimized for Apple Silicon (M1/M2/M3)
Qwen	Open-source large language model by Alibaba Cloud, used for structured profile extraction
Sarvam AI	Indian AI company providing multilingual NLP services including translation and STT
2dsphere	MongoDB geospatial index type supporting sphere-surface distance calculations
Haversine	Mathematical formula for calculating great-circle distance between two GPS coordinate points
GeoJSON Point	Standard format for representing geographic coordinates as `[longitude, latitude]`
Factor	One of six independent matching dimensions: Skill, Location, Availability, Pay, Rating, Relevance
Grounding	Validation that an LLM-extracted numeric value actually appears in the source transcript
Top-k	Maximum number of results returned in a recommendation query (default: 10)
RBAC	Role-Based Access Control — authorization based on user roles (worker, employer, admin)
Guided flow	Step-by-step voice profile creation where one field is captured per recording
Quick flow	Single-recording voice profile creation where all fields are extracted at once
Cold start	When a worker has no ratings yet (ratingAvg = 0), rating factor returns None
Dual-role account	A single user account holding both worker and employer roles simultaneously
Partial assignment	Job state when some (but not all) required worker positions are filled

---

This document describes the LokRozgar AI system as implemented in the repository at commit date October 2026. All schema definitions, algorithms, architectural claims, and API contracts have been verified against the actual source code.